# Two Meanings of Fixed Point: Quantizer Equilibria, Arithmetic Encodings, and the Case for Typed Decomposition

Flyxion
Independent Researcher

Revision: 9 September 2026

**Abstract**

"Fixed point" can name a self-consistent value of an iterative quantizer or a scaled-integer representation with a fixed radix position. Confusing convergence with encoding can obscure what a replay or provenance claim actually guarantees. This note separates the two meanings, treats modular numerical software as a design analogy rather than an audited implementation, and proposes distinct Rust wrapper types and function signatures for the two roles. It also distinguishes byte integrity from evidence that companion artifacts belong to one consistent checkpoint. The type example prevents one specific class of accidental argument substitution; it does not prove convergence, measurement accuracy, or the completeness of a provenance record.

## 1 Introduction

The two meanings of fixed point answer different questions. A converged update asks whether applying a map again changes a parameter. A fixed-radix representation asks how a value is encoded. This note uses a schematic ternary quantizer and a proposed modular implementation to keep those questions separate. Reports of external audits motivated the discussion, but their artifacts and exact revisions are not supplied here; no empirical finding from those reports is asserted as a verified result in this revision.

## 2 The Quantizer's Fixed Point

Consider a ternary quantizer built around a single non-negative threshold parameter $a$. Weights above the threshold survive quantization; weights below it are zeroed. A natural way to set $a$ is not to fix it in advance but to let it emerge from the distribution of surviving weights themselves:

$$a = \mathbb{E}\big[\, |w| \;\big|\; |w| \geq a/2 \,\big].$$

Here the survivor set is defined by a threshold of $a/2$, but the survivors in turn determine the next estimate of $a$. This is a self-referential equation rather than a closed-form definition, and its natural solution method is iteration: guess a value of $a$, compute the conditional expectation of the survivors under that guess's threshold, use the result as the next guess, and repeat until the value stops changing. The value at which iteration stops, if it stops, is a fixed point of the update map in the ordinary dynamical-systems sense: a point $a^*$ satisfying $a^* = T(a^*)$ for the update operator $T$.

This is a Lloyd–Max-style self-consistency condition for a symmetric ternary construction with levels $-a, 0, +a$ and half-level decision boundaries. It assumes the survivor set and conditional expectation are defined; ties and degenerate inputs need explicit policies. Its fixed point is a statement about where an iterative estimation procedure settles, not about how a number is stored. Two different quantizers with the same alphabet size can have different fixed points, because the fixed point depends on the distribution of the weights being quantized. There is no radix position, no scaling factor, and no encoding convention implied by the phrase in this context; there is only a converged parameter of a data-dependent update rule.

## 3 The Arithmetic Fixed Point

In fixed-point arithmetic, a value is represented as a scaled integer with an implicit radix position. For this example, define signed Q16.16 to mean a 32-bit signed integer $n$ representing

$$\widehat{v} = n/2^{16}.$$

Encoding a real-valued input requires a specified rounding rule and range policy; in general $\widehat{v}$ approximates that input. Recovering the represented value is exact as a rational expression, not a guarantee that the original input is recovered exactly.

Bit width and scale alone do not define arithmetic operations. Multiplication needs an intermediate width, rescaling and rounding rules, and overflow behavior; serialization needs byte order. These choices, together with event order and external inputs, define a replay contract. Fixed-point arithmetic can support reproducibility under that contract, but it does not guarantee reproducibility by itself, nor does the use of floating point make reproducibility impossible under every fully specified execution model.

## 4 Why the Conflation Is Costly

A quantizer equilibrium is a property of a map and its input distribution; a scaled-integer format is a representation chosen by a designer. Numerical stopping within a tolerance must also be distinguished from a proof that $a^* = T(a^*)$ exactly. An empty survivor set, tie handling, convergence failure, and overflow all require explicit treatment before the schematic update becomes a usable algorithm.

Provenance has a related distinction. Suppose a weight artifact and an event-history artifact each match their recorded digest, but they were captured at different update positions. Successful digest verification alone does not establish that the pair is a consistent checkpoint. The record should bind the artifacts to a common checkpoint identifier, their respective digests, the specification version, and the acquisition protocol. Even such a binding records a claim about consistency; validating it requires a trusted capture procedure, replay, or another declared check.

A cryptographic digest supports an integrity check against a trusted reference under the hash function's security assumptions. It does not by itself establish authorship, capture time, or consistency with companion files. This is a constructed failure scenario, not a claim that an identified external system was observed to fail in this way.

## 5 An Architectural Analogy: Typed Decomposition

A modular numerical application provides a useful design analogy. One can separate physical models, numerical updates, serialization, provenance, and rendering, then constrain the interfaces through which they interact. Crate boundaries alone do not prove isolation: shared mutable state, callbacks, capabilities, and side effects still need review.

For a symbolic ternary system, the proposed responsibilities are:

- *symbol*: protocol-level symbols and their declared meanings;
- *ternary*: the three-valued alphabet, independent of its byte encoding;
- *wire*: canonical bytes, widths, byte order, and validation;
- *quantizer*: estimation, convergence criteria, and failure outcomes;
- *envelope*: explicit bindings among history, parameters, and checkpoint artifacts;
- *replay*: transition order and recorded external inputs;
- *display*: observational projections whose mutation capabilities are restricted; and
- *audit*: checks of the declared integrity and correspondence obligations.

Concrete types and access boundaries establish the intended guarantee. Module names alone do not. Interaction between quantizer and wire modules requires conversions with explicit rounding, scaling, and failure policies.

## 6 A Contrast Worth Preserving

Numerical fidelity and byte-identical replay are different objectives. A physical simulation may be evaluated by error, stability, and preservation of relevant structure over time; a canonical protocol may additionally require exactly matching bytes across implementations. Neither objective automatically establishes the other. A deterministic simulation can reproduce a modeling error exactly, while a numerically useful approximation can fail a byte-replay contract because the latter leaves implementation choices unspecified.

These are compatible goals when both are explicitly designed and checked. No inference about a particular astrophysics codebase's integrator or reproducibility follows merely from its subject matter or crate organization.

## 7 A Typed Resolution

A sum type can label the alternatives, but a function accepting that sum type still accepts either variant. To enforce distinct argument roles, give the roles distinct wrapper types and use those types in function signatures:

```
#[derive(Clone, Copy)]
struct Q16_16(i32);

#[derive(Clone, Copy)]
struct QuantizerParameter(f64);

fn encode_arithmetic(value: Q16_16) -> [u8; 4] {
    value.0.to_le_bytes()
}

fn quantizer_threshold(value: QuantizerParameter) -> f64 {
    value.0 / 2.0
}

fn main() {
    let arithmetic = Q16_16(65536);
    let parameter = QuantizerParameter(1.0);
    assert_eq!(encode_arithmetic(arithmetic), [0, 0, 1, 0]);
    assert_eq!(quantizer_threshold(parameter), 0.5);

    // Rejected by the compiler if uncommented:
    // encode_arithmetic(parameter);
}
```

The parameter wrapper deliberately does not claim that its value is a proven equilibrium. A production interface would need checked constructors and a separate convergence result if that property matters. The example establishes only that these two argument types are not implicitly interchangeable. Explicit conversion remains possible and must carry its own policy.

An optional `FixedPoint` enum with arithmetic and quantizer variants is useful for heterogeneous storage or dispatch. A function accepting `FixedPoint`, however, must inspect the variant or return an error. The enum is a labeled union, not a function-parameter restriction to one of its variants.

## 8 Conclusion

A quantizer fixed point and a fixed-radix representation deserve different names at their interfaces because they establish different properties. Distinct wrapper types can prevent accidental substitution; explicit transition semantics can support replay; checkpoint bindings can make consistency claims inspectable. Each mechanism has a limited obligation. None turns a converged parameter into an exact measurement, a digest into complete provenance, or a module boundary into proof of isolation.